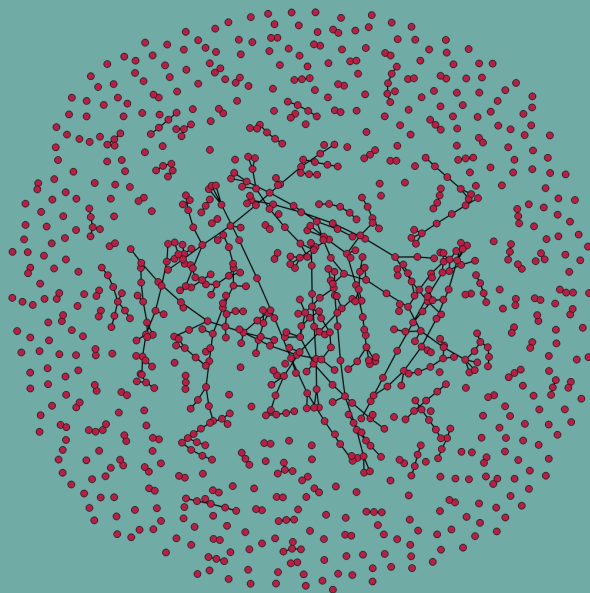


Thickness and Information in Dynamic Matching Markets

Mohammad Akbarpour, Shengwu Li,
Shayan Oveis Gharan

Senem Işık, Dorsa Majdi,
Thanawat Sornwanee



matching markets



Kidney Exchange



School Choice



Labor Markets



Dating

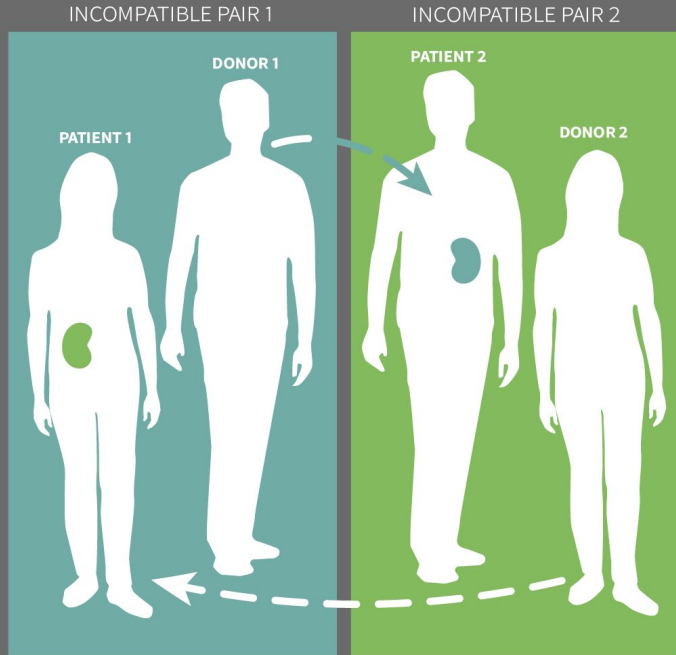


NRMP



Foster Care

motivation: kidney exchange market



This pair is
compatible with
some probability!

dynamics of the market: *a stochastic process*

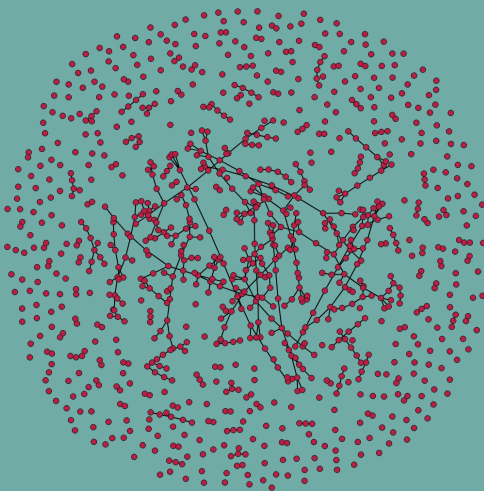
Any two agents are **compatible**
(there is an edge between
them) with probability p

Market

Agents **arrive** to the market
continuously with rate m



(number of arrivals $\sim$ Poisson(m))



Each agent gets **critical** (their last
time to match) with rate 1

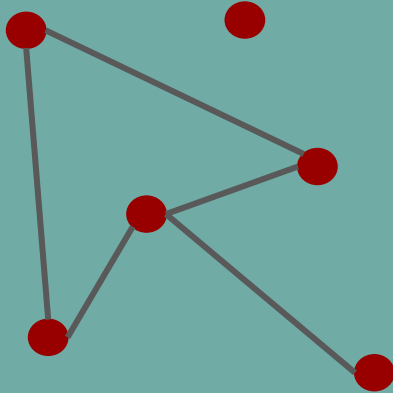
(Time it takes for an agent to get
critical $\sim$ Exponential(1))



Agents **depart** when matched or
critical and **perish**

Erdős-Rényi($|\mathbf{pool}|, p$)

model



If no one is matched:

- Expected number of agents waiting in the market = $\mathbf{m}$
(so over time horizon $\mathbf{T}$, it would be $\mathbf{m} \cdot \mathbf{T}$)
- Expected links a new agent makes upon arrival = $\mathbf{mp}$

$$\mathbf{d} = \mathbf{mp}$$

$\mathbf{d}$: expected degree, network *sparsity* parameter

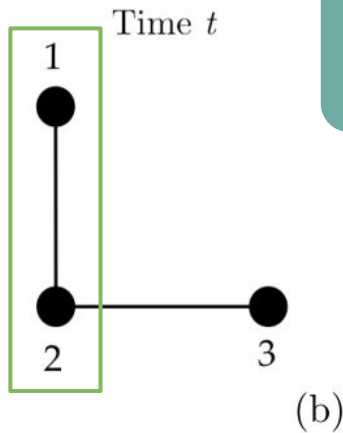
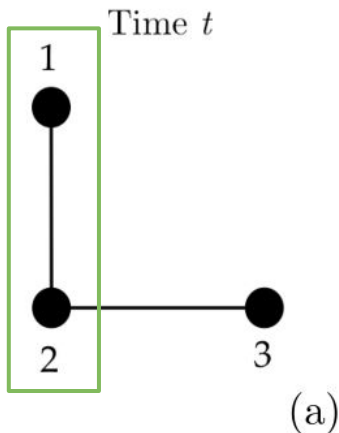
planner needs to decide *when* and *how* to **match online**

minimize **Loss** = expected fraction of **perished agents** (who leave unmatched 😞) for large T

$$\frac{\mathbb{E}[\text{number of agents who perish unmatched}]}{\mathbb{E}[\text{total number of agents who ever arrived}]} = \frac{mT - \mathbb{E}[|ALG(T) \cup \text{pool}_T|]}{mT}$$

observes $G(t)$ i.e. graph at time t **Challenge:** doesn't know the future...

Ignores the
cost of waiting



Thickening the
market helps!

what **information** does the planner have?

- (1) knows when the agents become critical.
- (2) does not know

a markov decision problem (MDP) $\rightarrow$ algorithm?

OMN (optimum omniscient algorithm in category (1))

1. would not match agents unless one of them is critical (waiting is weakly better)
2. would match critical agent to someone if possible (otherwise cannot save more than one agent...)
3. What if you have many neighbors? Which one?
 - a. would have optimized over the *whole network structure*
 - b. is a dynamic program with $\approx O\left(2^{\binom{m}{2}}/m!\right)$ states (**too complex!**)
 - c. but **is this necessary?**

simple local matching algorithms

Greedy Algorithm: match agents *upon their arrival* to a random neighbor (if any)

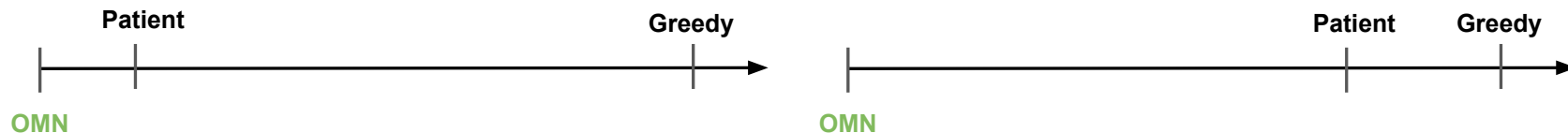
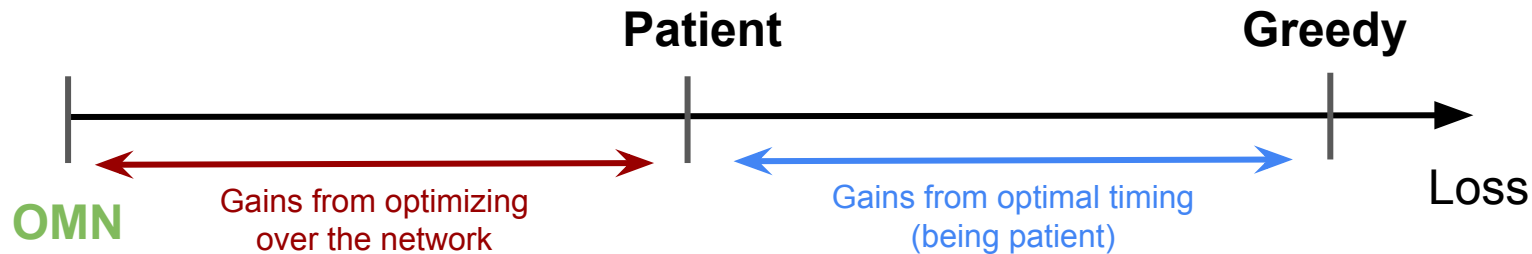
Patient Algorithm: match agents *when they get critical* to a random neighbor (if any)



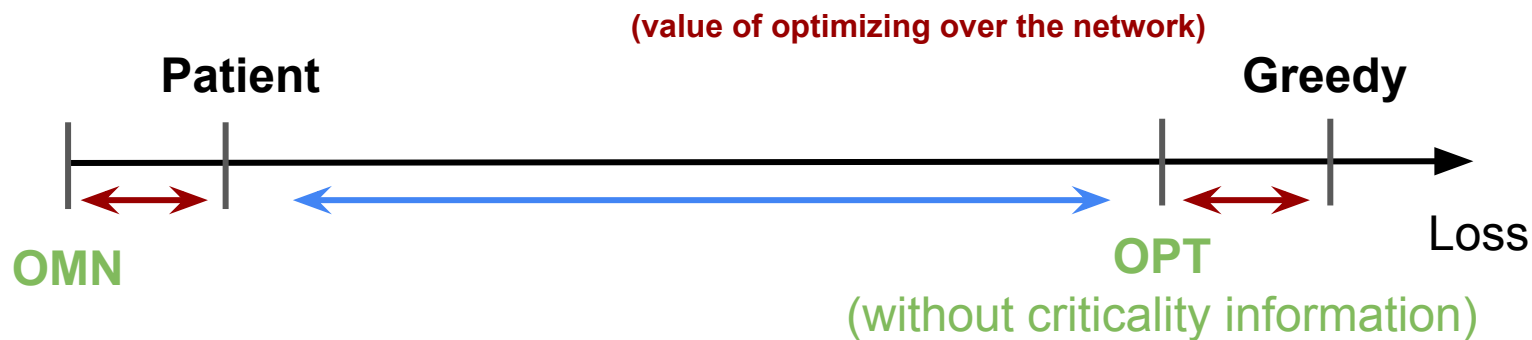
Smart in **when**, naive in **who**
(when there are ties, break arbitrarily)

Only algorithms that
know criticality can do
patient.

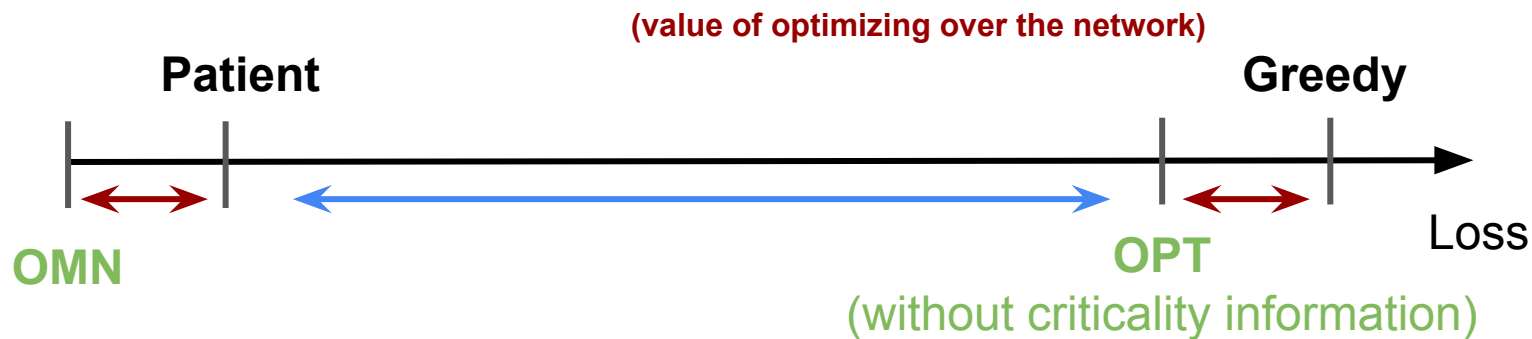
discussion: what would you expect?



results in a nutshell → value of (criticality) information



results in a nutshell $\rightarrow$ value of (criticality) information



Theorem 1. For (constant) $d \geq 2$, as $T, m \rightarrow \infty$,

$$\begin{aligned} L(\text{Greedy}) &\geq \frac{1}{2d+1} \\ L(\text{Patient}) &\leq \frac{1}{2} \cdot e^{-d/2} \end{aligned}$$

Optimizing over timing is huge.

proof ideas

Let $\mathbf{Z}(t)$ be the **pool size** i.e. the number of agents in the market waiting

Each agent becomes critical with rate 1

⇒ Under **Greedy**, *the rate of perishing* is $\mathbf{Z}(t)$

⇒ Under **Patient**, *the rate of perishing* is $\mathbf{Z}(t) (1 - p)^{\mathbf{Z}(t)} = \mathbf{Z}(t) (1 - d/m)^{\mathbf{Z}(t)}$

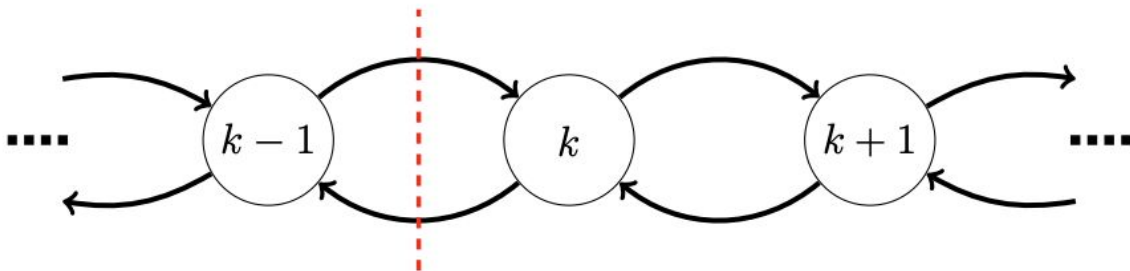
If we can give concentration results on $\mathbf{Z}(t)$ as t grows, we can characterize the expected loss of each algorithm!

Note the pool endogenously depends on the algorithm.

pool size is actually a Markov Chain!

and it is a **nice** Markov Chain!

- has a stationary distribution.
- converges to this distribution fast i.e. is mixing time is low.
- the stationary distribution concentrates on a value 😊



pool size is actually a Markov Chain!

and it is a **nice** Markov Chain!

- has a stationary distribution.
- converges to this distribution fast i.e. is mixing time is low.
- the stationary distribution concentrates on a value 😊

Lemma 11. For any $\epsilon > 0$, and $T > 0$,

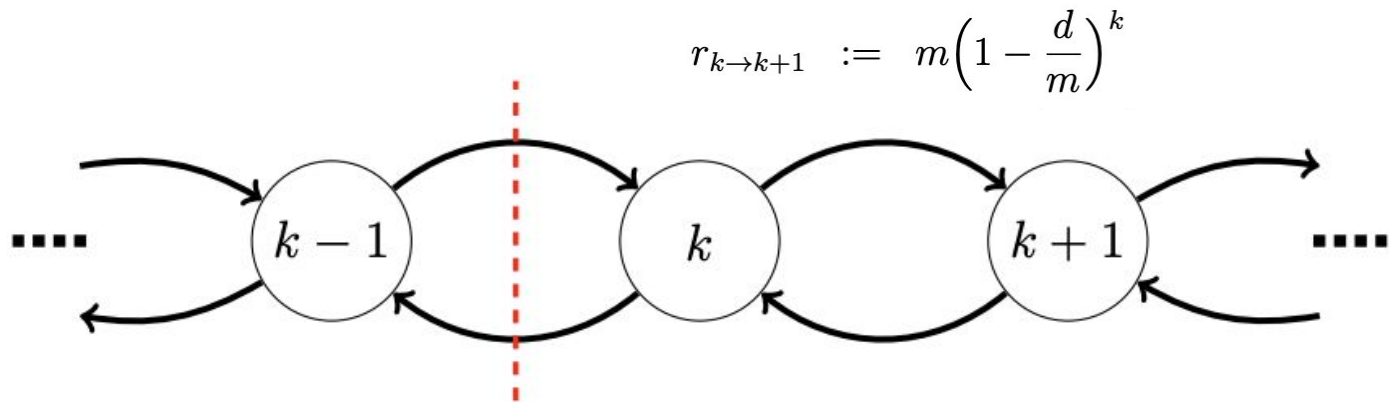
$$\mathbf{L}(\text{Greedy}) \leq \frac{\tau_{\text{mix}}(\epsilon)}{T} + 6\epsilon + \frac{1}{m}2^{-6m} + \frac{\mathbb{E}_{Z \sim \pi} [Z]}{m}.$$

Time it takes for
the markov
chain's
distribution to
come lops close to
the stationary
distribution
(in TV distance)

The upper bound
on how much the
pool could differ
from its
expectation under
stationary if its
distribution is lops
close to the
stationary

What we are going
to do next!

pool size under Greedy



$$r_{k \rightarrow k+1} := m \left(1 - \frac{d}{m} \right)^k$$

$$r_{k \rightarrow k-1} := k + m \left(1 - \left(1 - \frac{d}{m} \right)^k \right)$$

$$f(k) = r_{k \rightarrow k+1} - r_{k \rightarrow k-1}$$

pool size under Greedy

$$f(x) := m(1 - d/m)^x - (x + m(1 - (1 - d/m)^x)).$$

In particular, observe that $f(k) = r_{k \rightarrow k+1} - r_{k \rightarrow k-1}$. The above function is a decreasing convex function over non-negative reals. We define k^* as the unique root of this function. Let $k_{\min}^* := m/(2d + 1)$ and $k_{\max}^* := \log(2)m/d$. We show that $f(k_{\min}^*) \geq 0$ and $f(k_{\max}^*) \leq 0$. This shows that $k_{\min}^* \leq k^* < k_{\max}^*$.

$$f(k_{\min}^*) \geq -k_{\min}^* - m + 2m(1 - d/m)^{k_{\min}^*} \geq 2m\left(1 - \frac{k_{\min}^* d}{m}\right) - k_{\min}^* - m = 0,$$

$$f(k_{\max}^*) \leq -k_{\max}^* - m + 2m(1 - d/m)^{k_{\max}^*} \leq -k_{\max}^* - m + 2me^{-(k_{\max}^*)d/m} = -k_{\max}^* \leq 0.$$

remains to show **pool size** concentrates under stationary

Lemma 13. *For any integer $k \geq k^*$*

$$\frac{\pi(k+1)}{\pi(k)} \leq e^{-(k-k^*)d/m}.$$

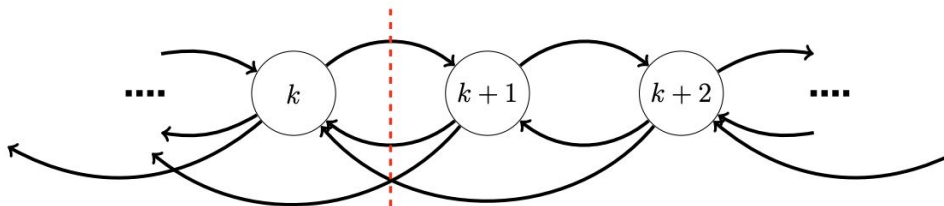
And, for any $k \leq k^$, $\pi(k-1)/\pi(k) \leq e^{-(k^*-k+1)d/m}$.*

Key ingredient: detailed balance equation for stationary distribution

$$\sum_{i \in S, j \notin S} \pi(i) r_{i \rightarrow j} = \sum_{i \in S, j \notin S} \pi(j) r_{j \rightarrow i} \quad \rightarrow \quad \pi(k-1) r_{k-1 \rightarrow k} = \pi(k) r_{k \rightarrow k-1}$$

This is the first time we are using that this is a stationary distribution! And this allows to give bounds on the pool size without knowing the stationary distribution itself (I think this is cool)

same story for **pool size** under patient



Lemma 15. For any $\epsilon > 0$ and $T > 0$,

$$\mathbf{L}(\text{Patient}) \leq \frac{1}{m} \mathbb{E}_{Z \sim \pi} [Z(1 - d/m)^{Z-1}] + \frac{\tau_{\text{mix}}(\epsilon)}{T} + \frac{\epsilon m}{d^2}$$

$$m\pi(k) = (k+1)\pi(k+1) + (k+2)\left(1 - \left(1 - \frac{d}{m}\right)^{k+1}\right)\pi(k+2) \quad (\text{E.5})$$

Now define a continuous $f : \mathbb{R} \rightarrow \mathbb{R}$ as follows,

$$f(x) := m - (x+1) - (x+2)(1 - (1 - d/m)^{x+1}). \quad (\text{E.6})$$

It follows that

$$f(m-1) \leq 0, f(m/2-2) > 0,$$

which means that $f(\cdot)$ has a root k^* such that $m/2 - 2 < k^* < m$. In the rest of the proof we show that the states that are far from k^* have very small probability in the stationary distribution

In order to complete the proof of **Proposition 16**, we first prove the following useful lemma.

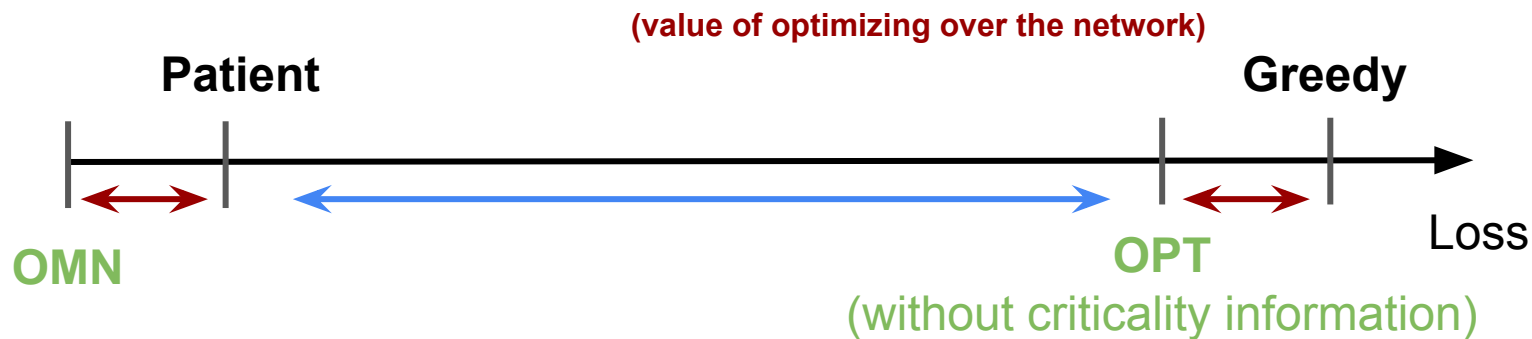
Lemma 25. For any integer $k \leq k^*$,

$$\frac{\pi(k)}{\max\{\pi(k+1), \pi(k+2)\}} \leq e^{-(k^*-k)/m}.$$

Similarly, for any integer $k \geq k^*$, $\frac{\min\{\pi(k+1), \pi(k+2)\}}{\pi(k)} \leq e^{-(k-k^*)/(m+k-k^*)}$

Key ingredient

results in a nutshell $\rightarrow$ value of (criticality) information

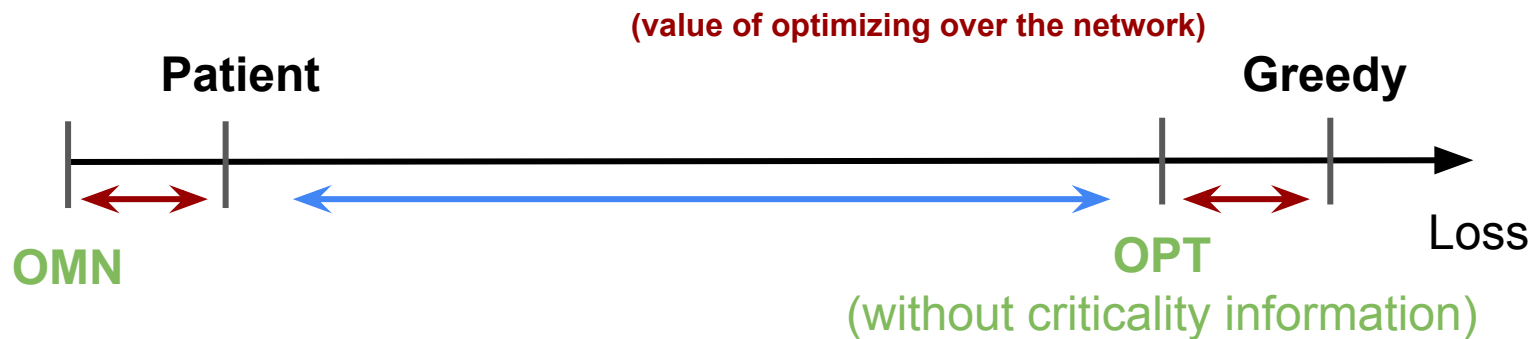


Theorem 1. For (constant) $d \geq 2$, as $T, m \rightarrow \infty$,

$$\begin{aligned} L(\text{Greedy}) &\geq \frac{1}{2d+1} \\ L(\text{Patient}) &\leq \frac{1}{2} \cdot e^{-d/2} \end{aligned}$$

Optimizing over timing is huge.

results in a nutshell $\rightarrow$ value of (criticality) information

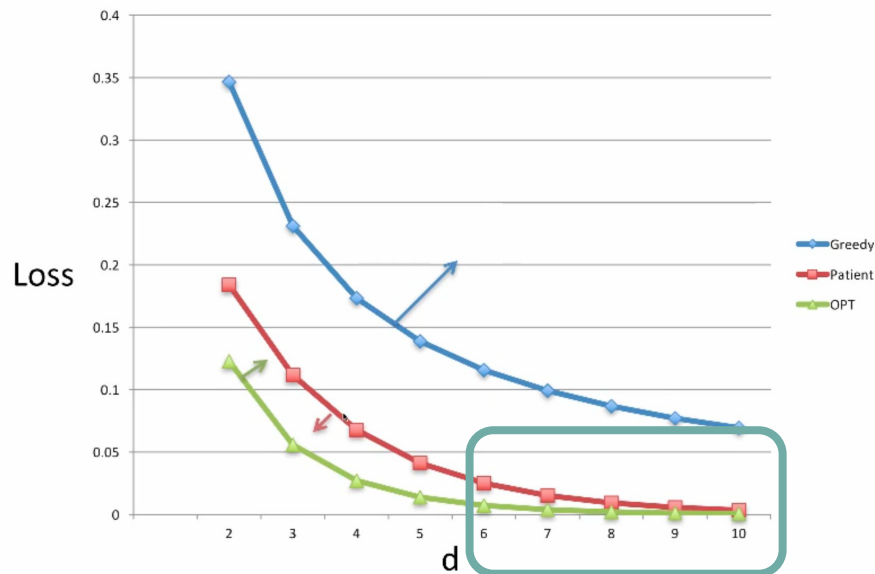


Theorem 2. Consider any algorithm ALG that observes the set of critical agents. Then, for (constant) $d \geq 2$, as $T, m \rightarrow \infty$,

Patient is close to optimal.

$$\mathbf{L}(\text{OPT}^c) \geq \frac{e^{-\frac{d}{2}(1+\mathbf{L}(\text{ALG}))}}{d+1}.$$

Recall that $\mathbf{L}(\text{Patient}) \leq \frac{1}{2} \cdot e^{-d/2}$. Substituting for $\mathbf{L}(\text{ALG})$ implies that¹² $\mathbf{L}(\text{OPT}^c) \geq \frac{e^{-\frac{d}{2}(1+e^{-d/2}/2)}}{d+1}$.



Market thickens
(as d increases)
solves the
network
optimization
problem

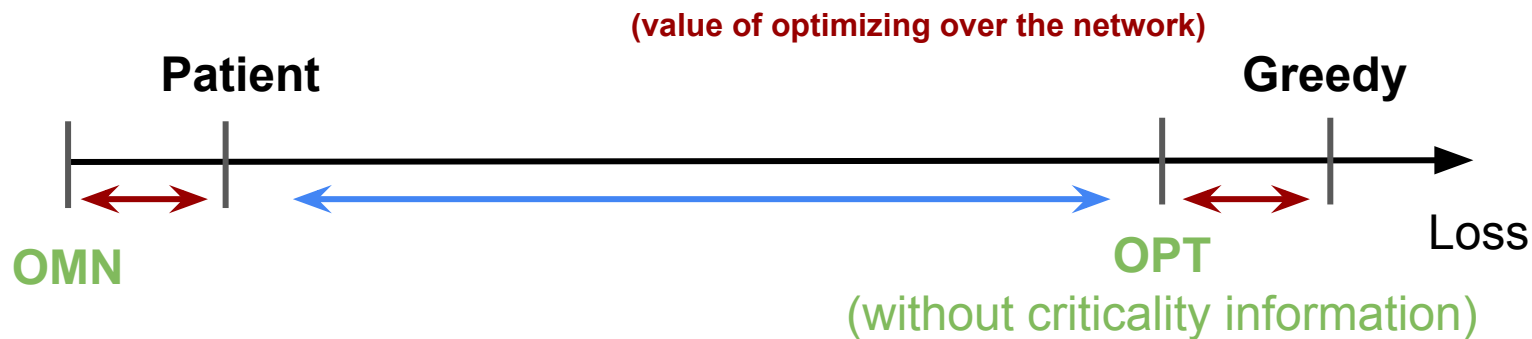
Theorem 2. Consider any algorithm ALG that observes the set of critical agents. Then, for (constant) $d \geq 2$, as $T, m \rightarrow \infty$,

Patient is close to optimal.

$$\mathbf{L}(\text{OPT}^c) \geq \frac{e^{-\frac{d}{2}(1+\mathbf{L}(\text{ALG}))}}{d+1}.$$

Recall that $\mathbf{L}(\text{Patient}) \leq \frac{1}{2} \cdot e^{-d/2}$. Substituting for $\mathbf{L}(\text{ALG})$ implies that¹² $\mathbf{L}(\text{OPT}^c) \geq \frac{e^{-\frac{d}{2}(1+e^{-d/2})}}{d+1}$.

results in a nutshell → value of (criticality) information



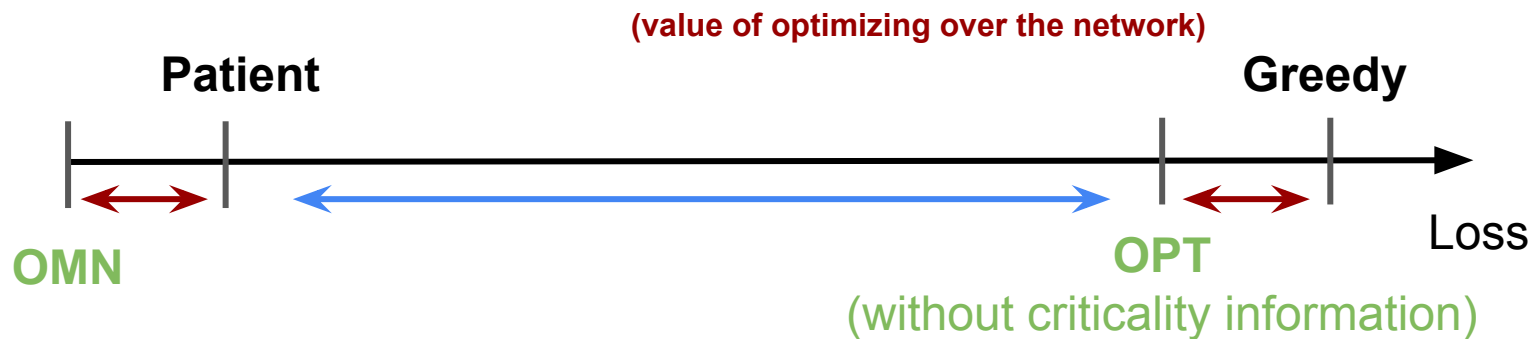
Theorem 3. For any DAG algorithm, and for (constant) $d \geq 2$, as $T \rightarrow \infty$,

$$L(DAG) \geq \frac{1}{2d+1}$$

DAG = Departure-Aware Greedy Algorithms

Any matching algorithm that does not wait, even with information, cannot come close to Patient.

results in a nutshell → value of (criticality) information



Theorem 4. For (constant) $d \geq 2$, as $T, m \rightarrow \infty$,

$$\frac{1}{2d+1} \leq \mathbf{L}(\text{OPT}) \leq \mathbf{L}(\text{Greedy}) \leq \frac{\log(2)}{d}$$

$$\frac{e^{-d}}{d+1} \leq \mathbf{L}(\text{OMN}) \leq \mathbf{L}(\text{Patient}) \leq \frac{1}{2} \cdot e^{-d/2}.$$

Waiting and criticality information are *complements*: waiting to thicken the market is substantially valuable only when the planner can identify urgent cases.

Our proposal



This pair is
compatible with
some probability!

The model assumes that agents are *ex ante* homogeneous, but in reality, agents may differ in how easy they are to match. Suppose *hard-to-match* (*H*) and *easy-to-match* (*E*) with compatibility probabilities:

$$p(HH) < p(HE) = p(EH) < p(EE)$$

If a critical agent faces both an *H* and an *E* neighbor, *ceteris paribus*, the optimal policy would match the *H*-type first, since *H*-types are harder to match and more likely to perish if left unmatched.

Empirical evidence (Liu et al., 2018, DiDi carpooling) suggests that when heterogeneity is moderate, the Patient algorithm still performs best, but with strong heterogeneity, Greedy can outperform it.

“Smart in **when**, naive in **who**” is not enough
(when there are ties, break arbitrarily)

Main difference: p and q are fixed whereas, before, they were vanishing with m as $p = d/m$. Ashlagi et. al. analyzed the **dense** regime.

Matching in Dynamic Imbalanced Markets

Itai Ashlagi

Afshin Nikzad

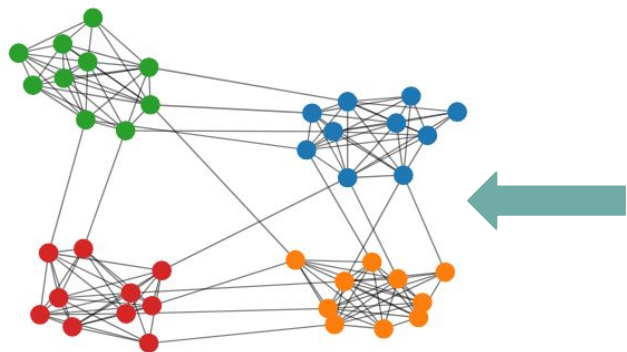
Philipp Strack*

Abstract

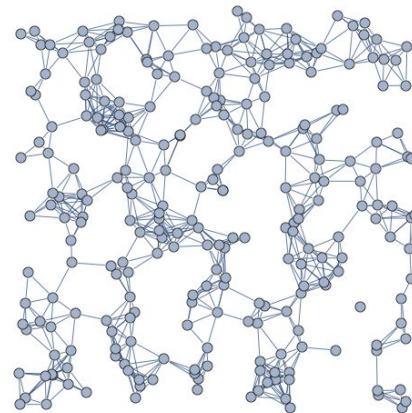
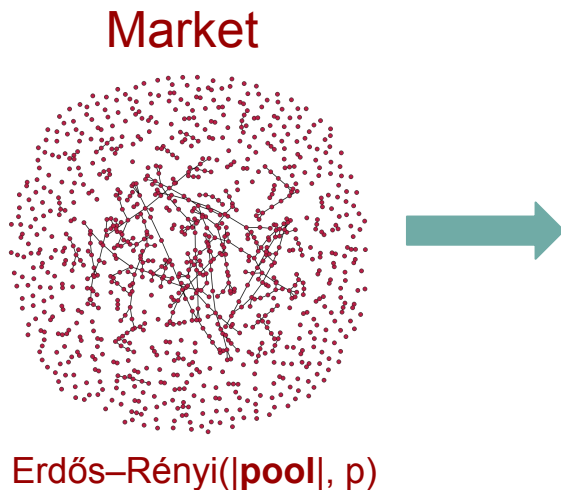
We study dynamic matching in exchange markets with easy- and hard-to-match agents. A greedy policy, which attempts to match agents upon arrival, ignores the positive externality that waiting agents provide by facilitating future matchings. We prove that the trade-off between a “thicker” market and faster matching vanishes in large markets; the greedy policy leads to shorter waiting times and more agents matched than any other policy. We empirically confirm these findings in data from the National Kidney Registry. Greedy matching achieves as many transplants as commonly-used policies (1.8% more than monthly batching), and shorter waiting times (16 days faster than monthly batching).

Our proposal: model?

This heterogeneity can be modeled as a **Stochastic Block Model (SBM)** (left): a random network where nodes belong to *blocks* (types) and link probabilities depend on block pairs.



In ride-sharing settings, agent compatibility depends on location too. A realistic extension is a **Geometric Random Graph** that places agents as points in space and connects two agents if their road-time distance is below a threshold.



Our proposal: **simple** algorithms?

- (1) **Type-Aware Greedy algorithm:** greedy but prioritizes hard-to-match agents, breaking ties arbitrarily.
- (2) **Type-Aware Patient algorithm:** waits but prioritizes hard-to-match agents, breaking ties arbitrarily.
- (3) **Type-Aware Hybrid algorithm:** match any H the moment it receives a neighbor and be patient otherwise.

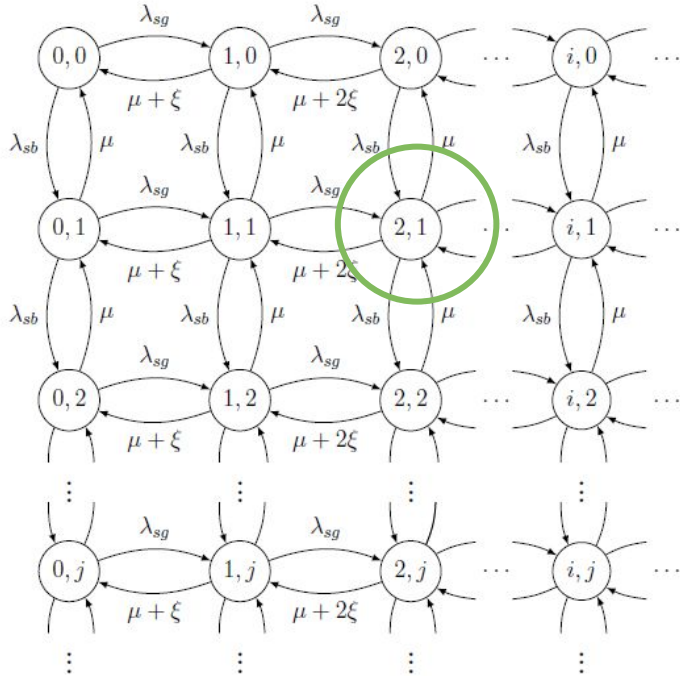
Recall that the pool endogenously depends on the algorithm and determines the loss of the system. How do we expect the steady-state composition of the pool under each strategy? **Some obvious ones:**

- (1) Type-Aware **Patient** algorithm: stochastic block model
- (2) Type-Aware **Greedy** algorithm: edgeless
- (3) Type-Aware **Hybrid** algorithm: isolated H-type vertices and Erdos-Renyi between E-types with parameter $p(EE)$.

What if we have multiple types but **can not observe** them?

Local but slightly more sophisticated algorithm: Do greedy or patient, but match to the node with **the lowest degree**.

algorithms: **difficulty** in the analysis



Now, we have two dimensional Markov chain for the pool size: one for number of **easy** types and one for number of **hard** types.

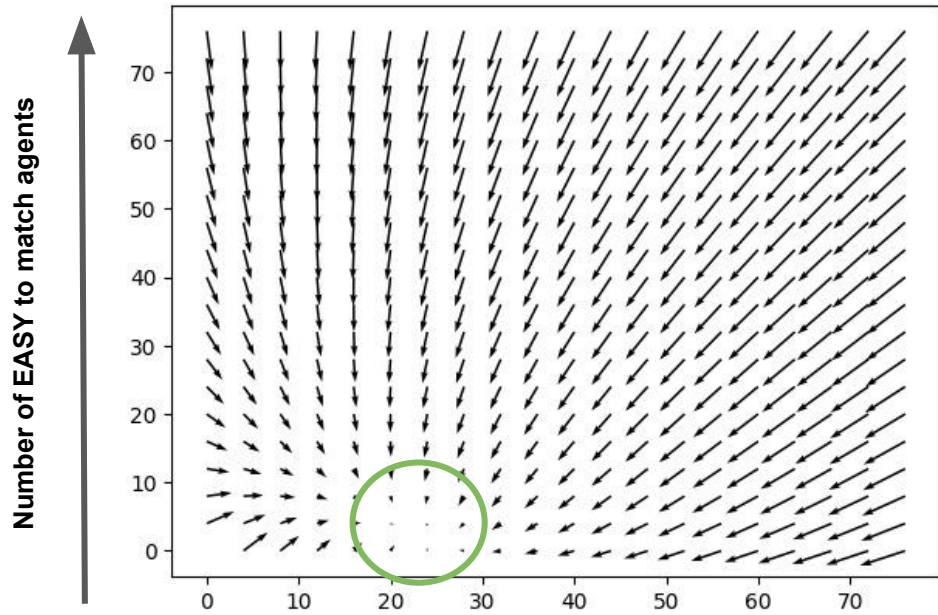
It is hard to find a state k^* such that the rates will cancel out each other analytically.

However, according to **simulations**, we see a state in which rates cancel out!

simulations: Markov Chain under Greedy

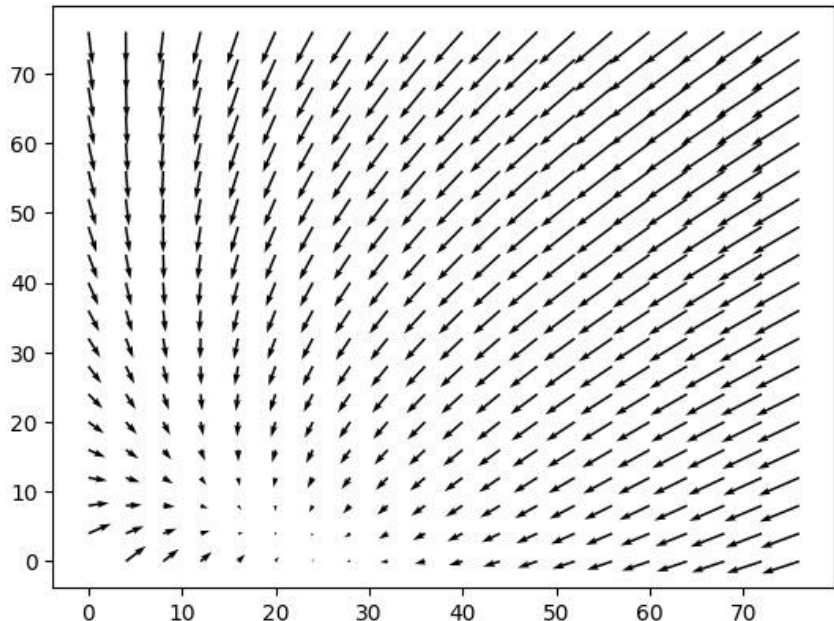
Type-Aware algorithms decreases hard to match agents more, as expected.

Type-Unaware



Number of EASY to match agents

Type-Aware



Number of HARD to match agents

At equilibrium, easy to match agents disappear, as expected.

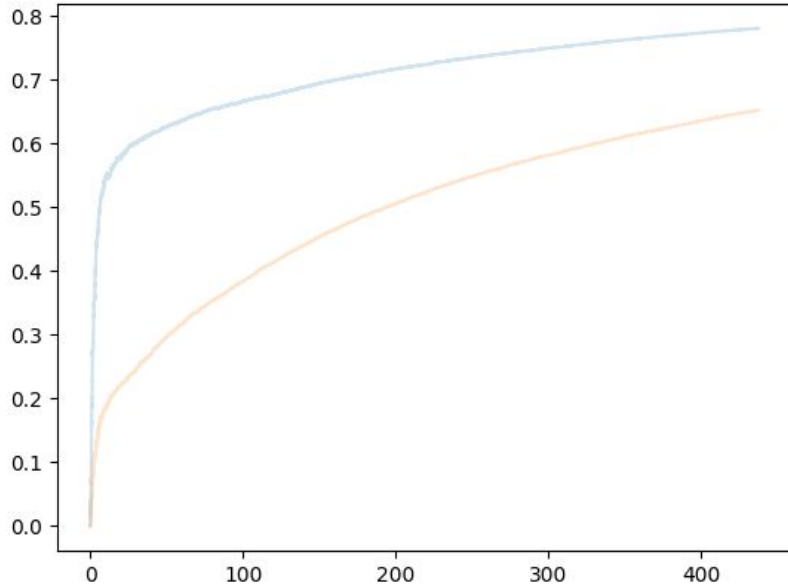
$m = 200.2$
Hard:40% of arrivals, Easy:60% of arrivals
 $p_{HH} = 0.008$, $p_{HE} = p_{EH} = 0.023$, $p_{EE} = 0.059$
(sparse regime)

simulations: performance of Patient

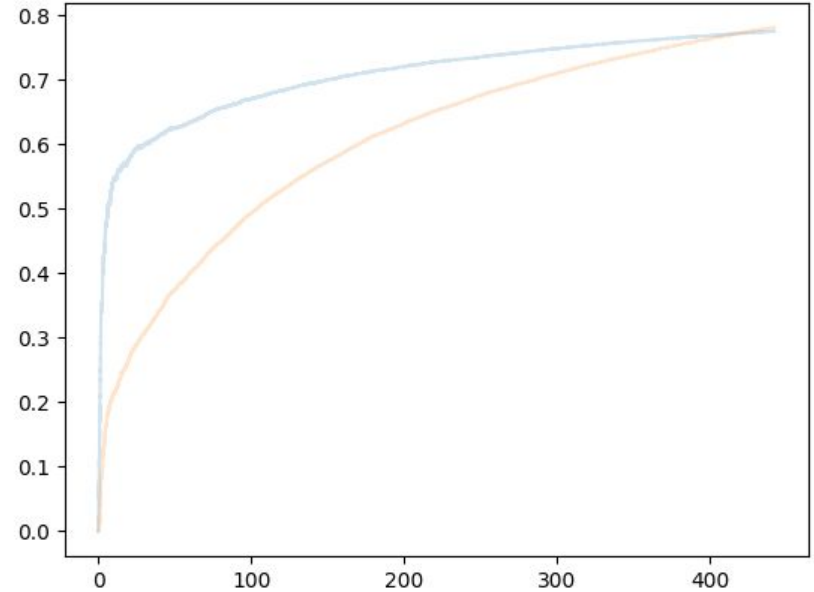
Ratio of EASY-to-match MATCHED

Ratio of HARD-to-match MATCHED

Type-Unaware



Type-Aware

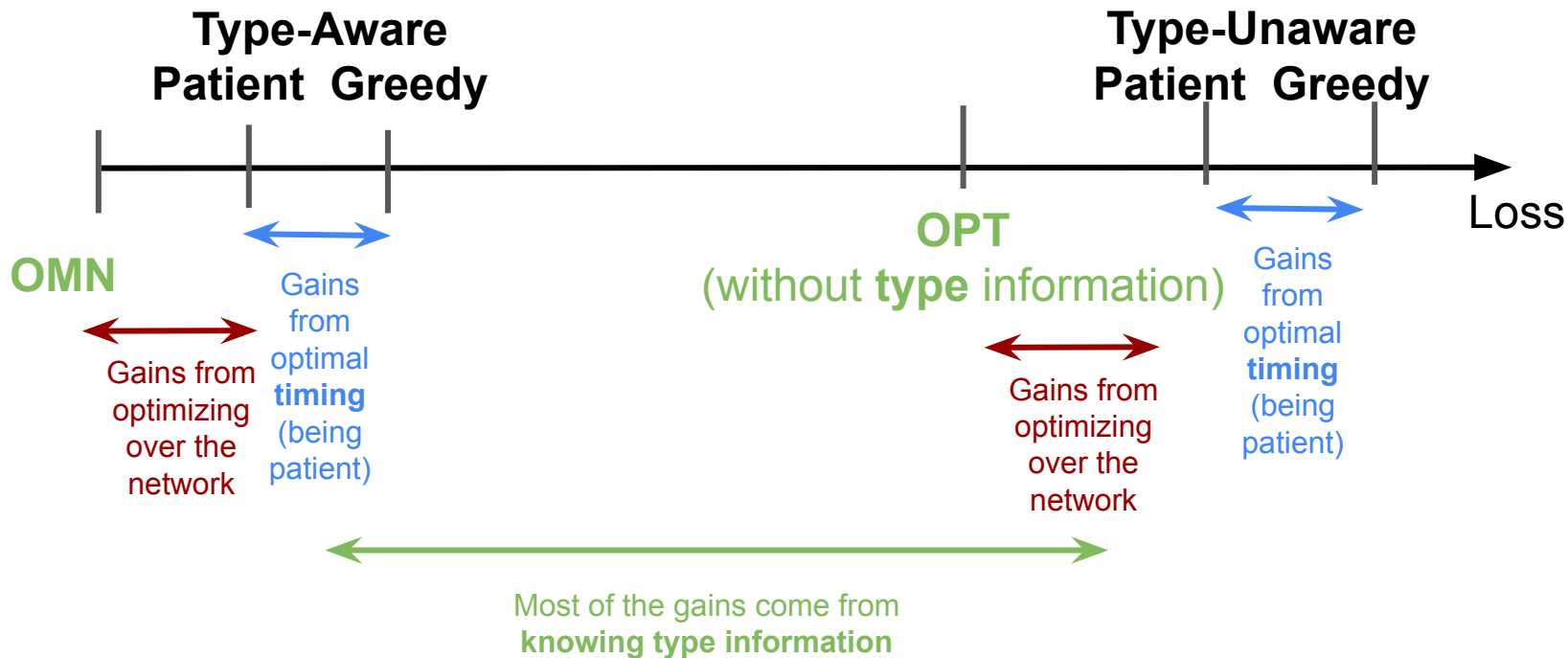


Both algorithms save around the same number of EASY agents but Type-Aware algorithm is able to save more HARD agents. (Note we didn't run these until convergence)

$m = 700.6$
Hard:40% of arrivals, Easy:60% of arrivals
 $d_{HH} = 0.06$, $d_{EH} = 0.3$, $d_{EE} = 1$
(note $d_{EE}=1$, for different parameters we would see different behaviour...)

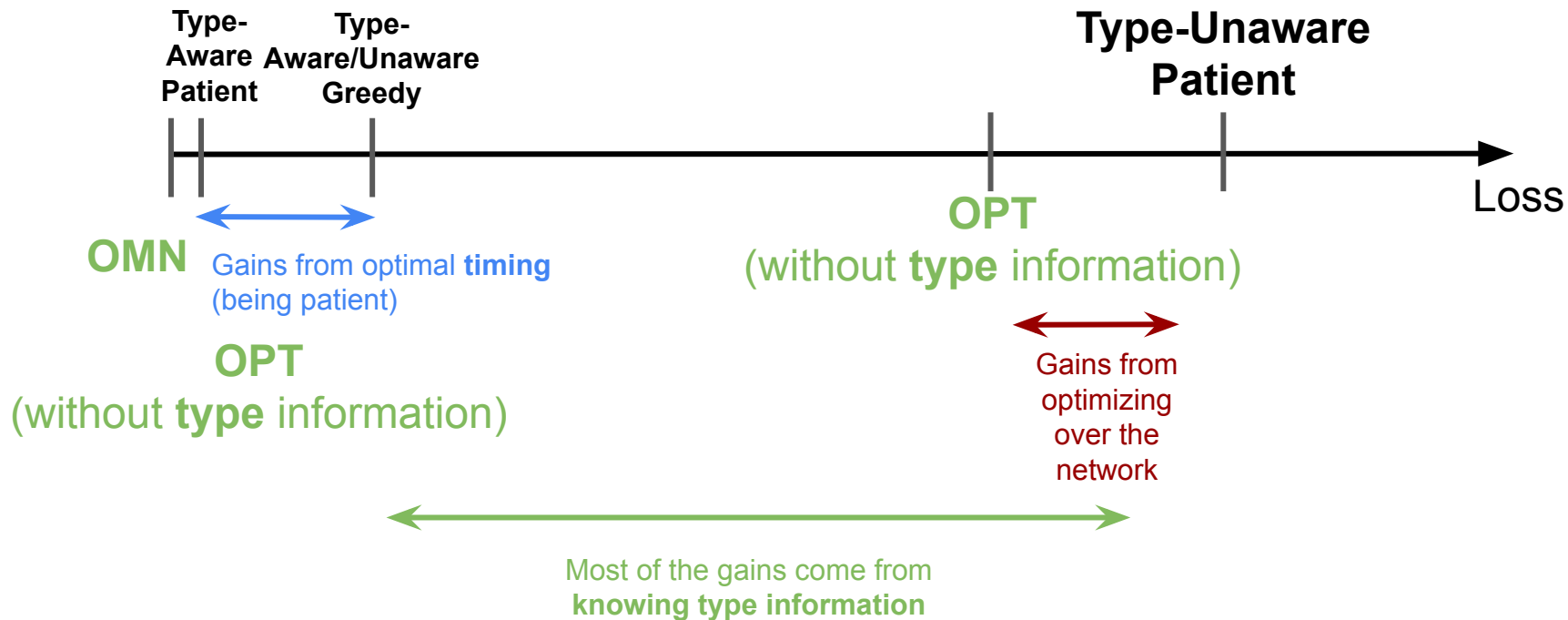
it would be interesting to show... in heterogeneous case

“who” matters more than “when” but still we don’t need optimize over the whole network structure



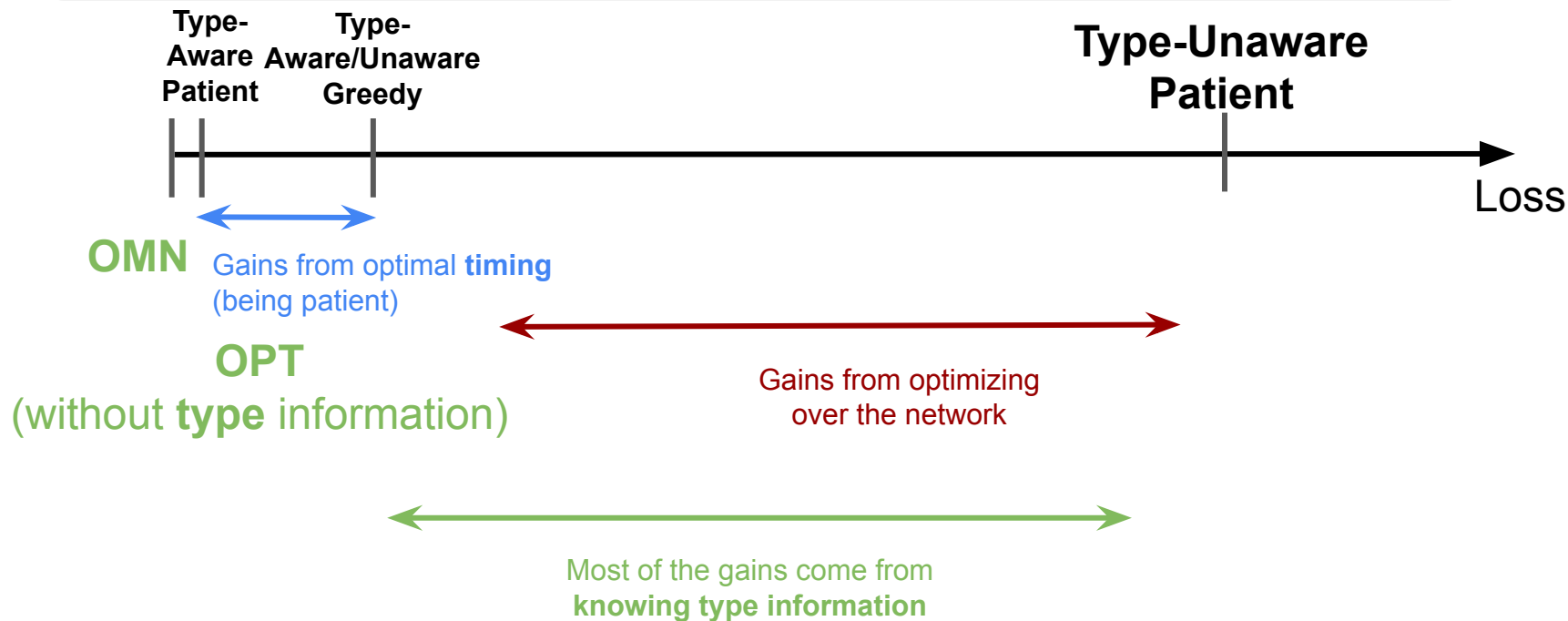
example

Heterogeneous markets behave in unintuitive ways in **greedy** setting.

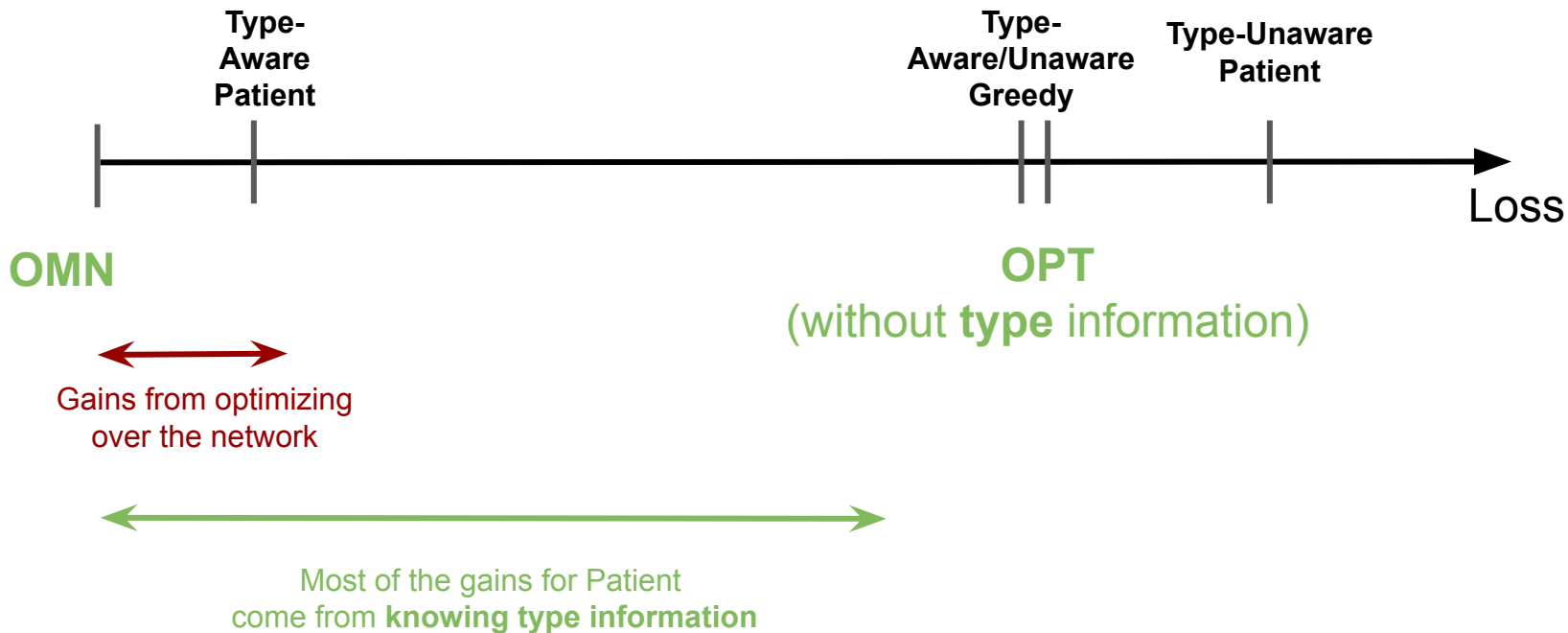


conjecture

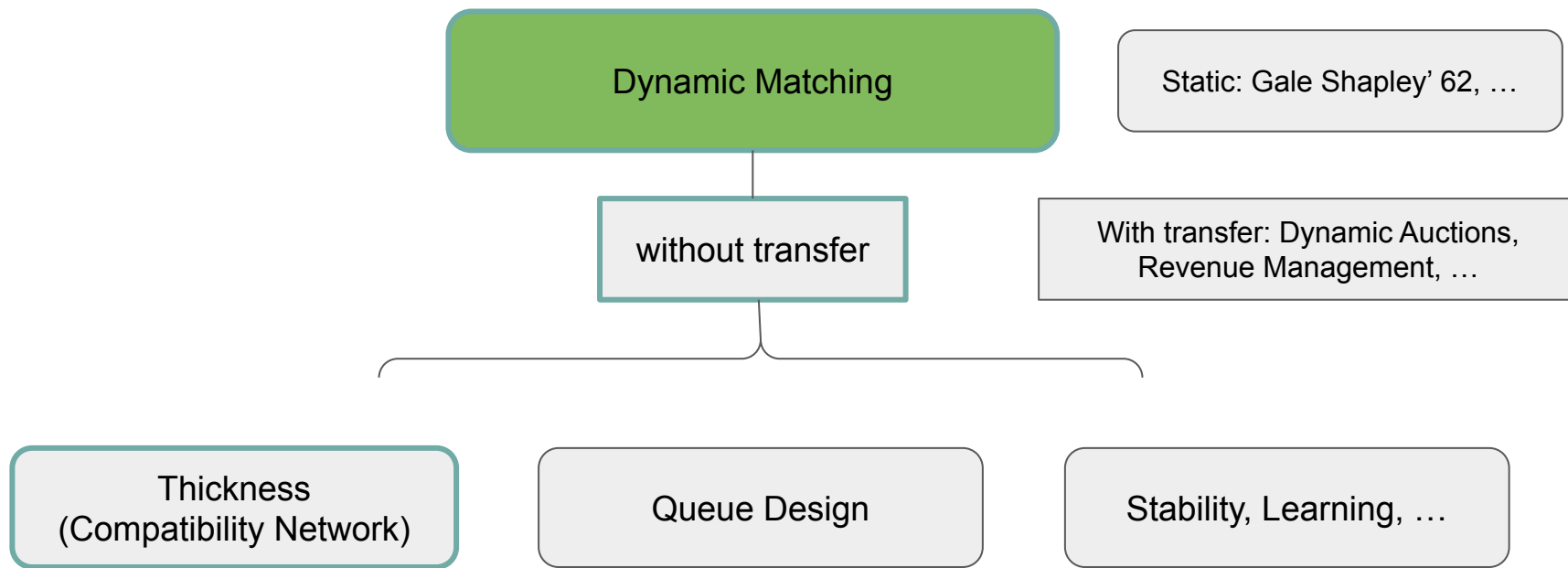
note **degree** of the node is a good enough **estimator** for how hard it is to match it... as m grows...



conjecture



finally: **literature review**



(motivated by Kidney exchange)

Thickness in Dynamic Matching: before and after

Dynamic Kidney Exchange
(Ünver 2010)

Introduce compatibility-based pref
=> Delay matching for thickness

Ashlagi, Jaillet, and
Manshadi (2013)

Sparsity of Kidney exchange network
Random Graphs
Sparse <> wait to thicken

Ashlagi and Roth (2014)

Anderson, Ashlagi,
Gamarnik, Kanoria (2017)

Ashlagi, Burq, Jaillet, and
Manshadi (2018)

Akbarpour et al (2020)

Ashlagi, Nikzad, and
Strack (2023)

Ashlagi, Gurvich,
Kerimov (2024)

No stochastic departure
Efficiency <> avg wait time
AAGK: Greedy ~ OPT

Value of departure information
Match rate

Two types
Dense Network

Type
“All-time-regret”
Modified Greedy ~ OPT

Waiting time and match rate: Greedy ~ OPT

Where our proposal stands in the literature

Dynamic Matching with Stochastic Departure

Objective = Match rate

	Heterogeneous	Homogeneous
Dense graphs	Patient ~ Greedy ~ Optimal	
Sparse graphs	Type Aware >> Type unaware?	Patient >> Greedy

Thank you!

Slides for Mohammad

Our proposal



This pair is
compatible with
some probability!

The model assumes that agents are *ex ante* homogeneous, but in reality, agents may differ in how easy they are to match. Suppose *hard-to-match* (H) and *easy-to-match* (E) with compatibility probabilities:

$$p(HH) = 0 < p(HE) = p(EH) = d_1/2m < p(EE) = d_2/2m$$

If a critical agent faces both an H and an E neighbor, *ceteris paribus*, the optimal policy would match the H -type first, since H -types are harder to match and more likely to perish if left unmatched.

Empirical evidence (Liu et al., 2018, DiDi carpooling) suggests that when heterogeneity is moderate, the Patient algorithm still performs best, but with strong heterogeneity, Greedy can outperform it.

“Smart in **when**, naive in **who**” is not enough
(when there are ties, break arbitrarily)

Main difference: p and q are fixed whereas, before, they were vanishing with m as $p = d/m$. Ashlagi et. al. analyzed the **dense** regime.

Matching in Dynamic Imbalanced Markets

Itai Ashlagi

Afshin Nikzad

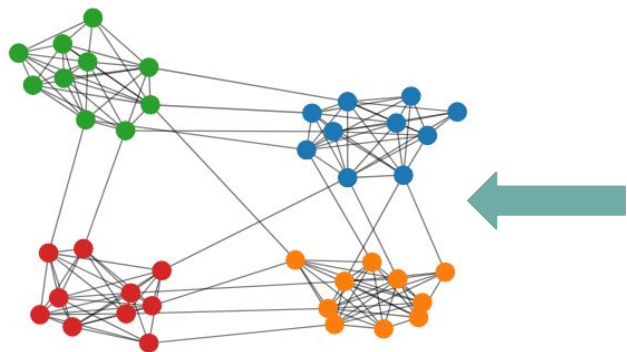
Philipp Strack*

Abstract

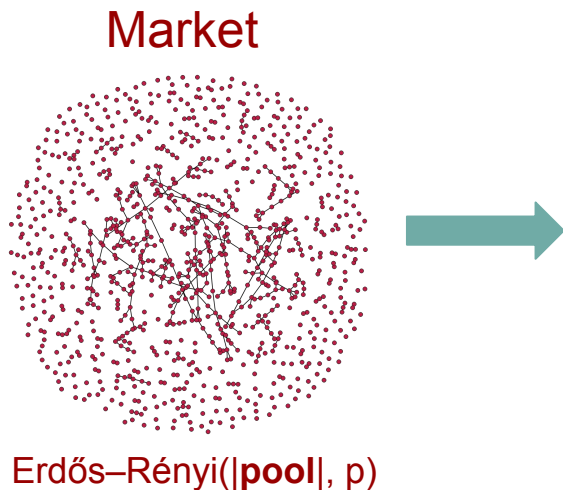
We study dynamic matching in exchange markets with easy- and hard-to-match agents. A greedy policy, which attempts to match agents upon arrival, ignores the positive externality that waiting agents provide by facilitating future matchings. We prove that the trade-off between a “thicker” market and faster matching vanishes in large markets; the greedy policy leads to shorter waiting times and more agents matched than any other policy. We empirically confirm these findings in data from the National Kidney Registry. Greedy matching achieves as many transplants as commonly-used policies (1.8% more than monthly batching), and shorter waiting times (16 days faster than monthly batching).

Our proposal: model?

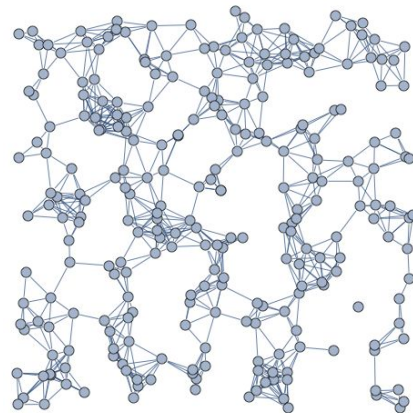
This heterogeneity can be modeled as a **Stochastic Block Model (SBM)** (left): a random network where nodes belong to *blocks* (types) and link probabilities depend on block pairs.



In ride-sharing settings, agent compatibility depends on location too. A realistic extension is a **Geometric Random Graph** that places agents as points in space and connects two agents if their road-time distance is below a threshold.



Erdős-Rényi($|\text{pool}|, p$)



Our proposal: **simple** algorithms?

- (1) **Type-Aware Greedy algorithm:** greedy but prioritizes hard-to-match agents, breaking ties arbitrarily.
- (2) **Type-Aware Patient algorithm:** waits but prioritizes hard-to-match agents, breaking ties arbitrarily.
- (3) **Type-Aware Hybrid algorithm:** match any H the moment it receives a neighbor and be patient otherwise.

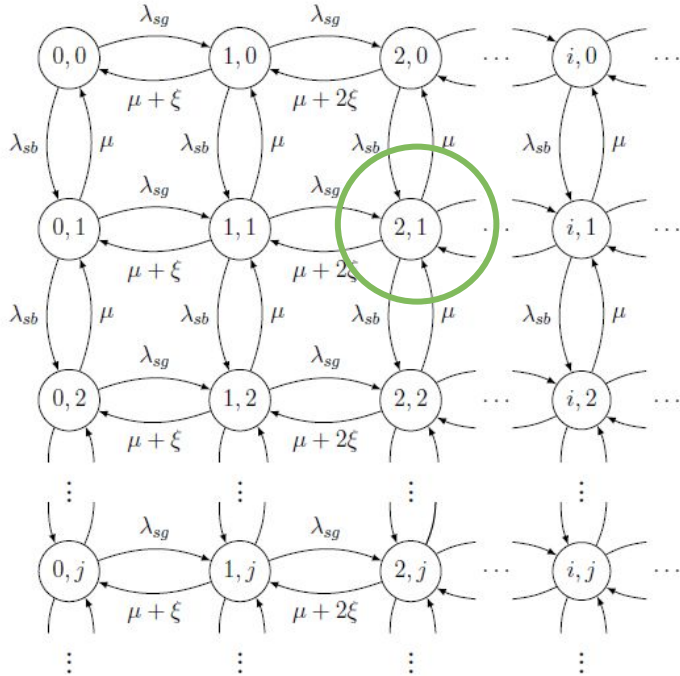
Recall that the pool endogenously depends on the algorithm and determines the loss of the system. How do we expect the steady-state composition of the pool under each strategy? **Some obvious ones:**

- (1) Type-Aware **Patient** algorithm: stochastic block model
- (2) Type-Aware **Greedy** algorithm: edgeless
- (3) Type-Aware **Hybrid** algorithm: isolated H-type vertices and Erdos-Renyi between E-types with parameter $p(EE)$.

What if we have multiple types but **can not observe** them?

Local but slightly more sophisticated algorithm: Do greedy or patient, but match to the node with **the lowest degree**.

algorithms: **difficulty** in the analysis

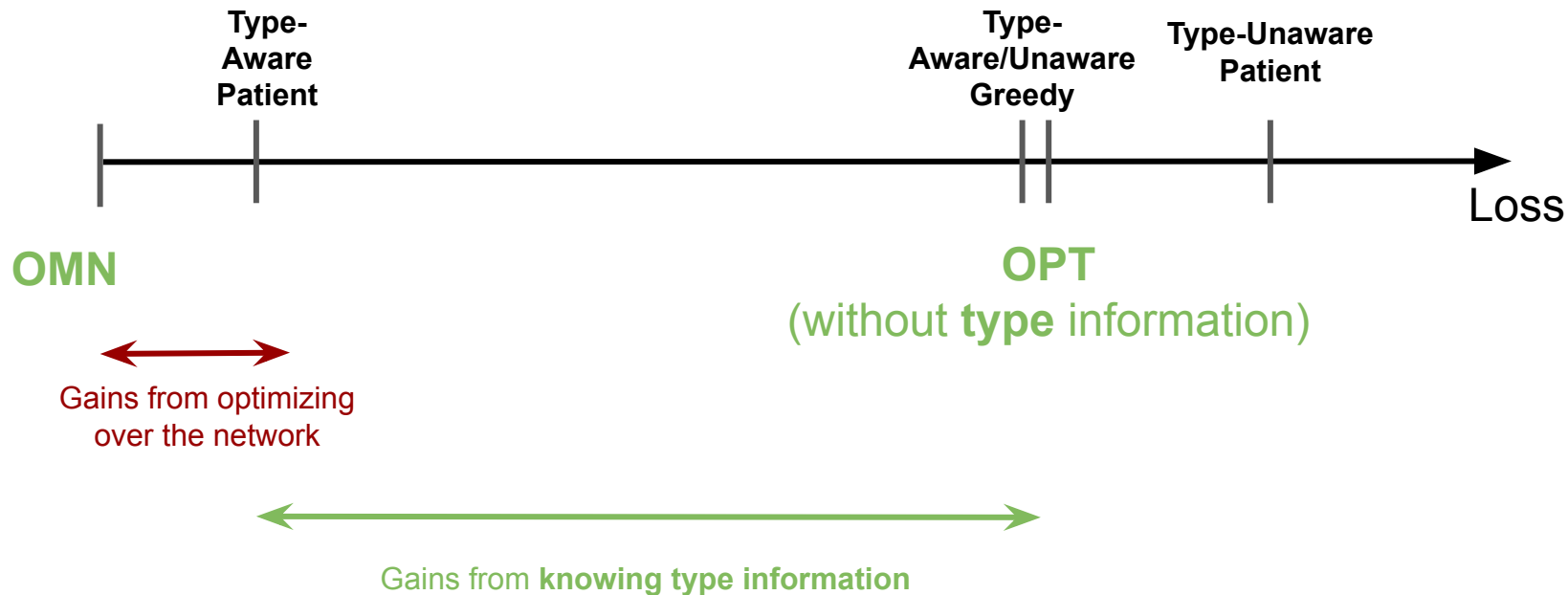


Now, we have two dimensional Markov chain for the pool size: one for number of **easy** types and one for number of **hard** types.

It is hard to find a state k^* such that the rates will cancel out each other analytically.

However, according to **simulations**, we see a state in which rates cancel out!

conjecture



Why is type aware/unaware greedy have the same performance?



Suppose E arrives. The type unaware policy will just match it to E with high probability.

If E exists in the pool the algorithm matches E's.

Eventually there are no E's in the pool.

E's trivially connect to H's!!!

takeaways



Pooling requires optimization.

Without type-aware selection, being Patient can be worse than being Greedy.

Under Patient the E agents wait and at criticality the abundant E–E channel pulls E criticalities toward E partners rather than H.

Waiting helps only when the matching rule deliberately uses the E pool to rescue H.

Future directions

Equal arrival rate

Equal perishing rate

Simulations